



17



2



2550 sp

عملي

كلية الهندسة المعلوماتية

السنة الثانية

التعامل مع الذاكرة باستخدام Assembly

م.عبير ميا



RB Informatcs; 14/04/2026

محتوى مجاني غير مخصص للبيع التجاري

بنیان الحاسوب ا

السلام عليكم ورحمة الله وبركاته

هل تكفي السجلات للتعامل عند البرمجة بلغة التجميع أم أننا نحتاج الذاكرة ؟
نعلم انه من ميزات لغة التجميع السرعة في التنفيذ إذا لما سنستخدم الذواكر طالما ان السجلات اسرع ؟
ذلك لان حجم السجلات محدود (وهو 32 سجل) وقد نحتاج للتعامل احيانا مع عدد كبير من البيانات التي لا
تتسع في السجلات فنلجأ حينها للذاكرة.

العنونة:

للعنونة نوعان:

1. العنونة بالبايت:

كل بايت له عنوان مستقل (فريد) ويكون عنوان الكلمة هو عنوان اول بايت فيها وكل تعليمة هي عبارة عن كلمة
عرضها 32 بت أي 4 بايت وتبعد كل تعليمة عن التعليمة التي تسبقها بمقدار اربعة بايت ومن مميزاته انه يمكننا
ان نصل الى كل بايت فيسهل علينا التعديل على المحارف او الكلمات.
مثال:

بفرض لدينا تعليمة مخزنة في العنوان 100 سيكون عنوان اول بايت فيها هو 100 وعنوان ثاني بايت 101
وثالث بايت 102 وهكذا

اما التعليمة الثانية ستبدأ من العنوان 104

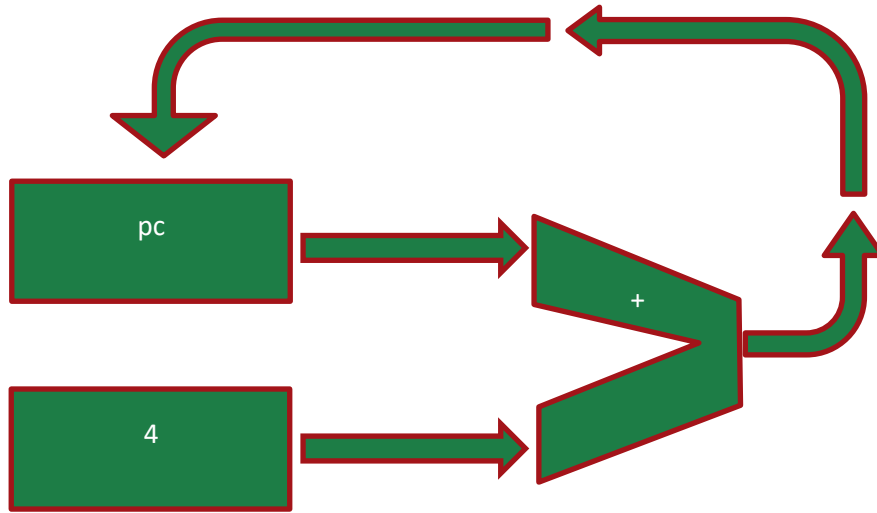
2. العنونة بالكلمة:

هنا لكل كلمة عنوان مستقل وجميع ال Bytes المكونة لها تشترك بهذا العنوان وكل كلمة تبعد عن التي تسبقها
بمقدار واحد بايت.

اذا بفرض ان لدينا تعليمة في العنوان 100 فان التعليمة التي تليها ستكون بالعنوان 101 و التي تليها 102 وهكذا

ملاحظة: : يوجد معالجات تعنون بالبايت واخرى تعنون بالكلمة لكن معالج MIPS الذي نتعامل
معه يعنون فقط بالبايت

الـ program counter (pc): وهو عبارة عن سجل يؤشر على التعليمة الحالية وهو عداد يحمل عنوان التعليمة التي يجب تنفيذها حالا ويزداد بمقدار اربعة كل مرة (حتى ينتقل الى التعليمة التالية) نمثله بالـ hardware بالشكل الاتي:



- في الـ MIPS لا يمكننا الجمع بين السجل و الذاكرة ولحل هذه المشكلة يجب تحميل البيانات من الذاكرة و العمليات الحسابية تتمها بالسجلات (لا يمكننا الاستعانة بالذاكرة للعمليات الحسابية و المنطقية لانها ستأخذ وقت اطول)
- في الذاكر من نوع MIPS اذا كان عدد خطوط العناوين هو 32 فيمكن ان نحجز $2^{32} = 4GB$ عنوان.
- تميز المبرمج بلغة التجميع يأتي من محاولته تقليل الوصول للذاكرة لأن الوصول لها مكلف.
- س : ما الذي يوجد في الذاكرة المعطيات/التعليمات/الاكواد؟
ج : كل شي يتعلق بالبرنامج.

الذاكرة في MIPS

🔗 يوجد طريقتين للتعامل مع الذاكرة:

- تعليمة (lw (load word : القراءة أو التحميل من الذاكرة
- تعليمة (sw (store word : الكتابة أو التخزين في الذاكرة

التحميل من الذاكرة:

وهي عملية النقل من الذاكرة للسجل ويتم ذلك باستخدام كلمة lw تعليمة التحميل

lw rt, offset(rs)

الـ offset: مقدار الانزياح وهو قيمة عددية (ويمثل البعد عن القاعدة)

الـ rs: وهو السجل القاعدي

تقوم التعليمة بجمع مقدار الانزياح offset مع محتوى السجل القاعدي rs, والناتج يكون عنوان في الذاكرة ويتم اخذ المحتوى في هذا العنوان وتحميله للسجل rt ونعبر عنه بهذا الشكل:

$\text{Mem}[\text{rs} + \text{offset}] \rightarrow \text{rt}$

نلاحظ ان العملية lw تحوي rt و rs ولا تحوي rd اذا فهي من النمط l-type

مثال:

Lw \$10, 12(\$5)

اذا التعليمة ستجمع محتوى السجل \$5 وليكن 1000 مع العدد 12 فينتج لدينا 1012 ثم نحمل محتوى هذا العنوان الى السجل \$10

التخزين في الذاكرة:

هو التخزين من السجل للذاكرة وتعليمة التخزين

sw rt, offset(rs)

لاحظ انها تشابه تعليمة التحميل وذلك لتسهيل التحكم, ولكي نحسب العنوان المطلوب نجمع محتوى السجل القاعدي rs مع القيمة offset لنحصل على عنوان الذاكرة المطلوب. ثم نقوم بتخزين محتوى السجل rt في ذلك العنوان. ونعبر عنها بالشكل

$\text{Mem}[\text{rs} + \text{offset}] \leftarrow \text{rt}$

يمكننا تحميل / تخزين بايت واحد من / في الذاكرة باستخدام التعليمتين lb sb (عند التعامل مع المحارف)

تمارين:

تمرين 1:

بفرض لدينا مصفوفة A تبدأ عند العنوان 0x10010000 المحفوظ في السجل \$t0 (مؤشر بدايتها), و اردنا الوصول للعنصر الخامس فيها A[4] لقراءته ووضعه في السجل \$s0, اكتب التعليمة اللازمة بلغة التجميع MIPS وماهي قيمة العنوان المطلوب قراءة محتواه ممثلاً بالنظام الست عشري؟

الحل:

$\text{M}[0\text{x}10010010] \leftarrow \text{\$s0}$

lw \$s0, 16(\$t0)

lw \$s0, 0x10(\$t0)

إذاً بدايةً نحتاج لقراءة محتوى فسنستخدم التعليمة lw السجل القاعدي وهو \$t0 وللوصول للعنصر الخامس أي لنتذكر أن العنصر الأول يبعد عن القاعدة صفر و الثاني اربعة والثالث ثمانية والرابع اثنا عشر أما الخامس فهو يبعد ستة عشر لذلك وضعنا الرقم 16 وهو 10 في النظام الست عشري.

■ ملاحظة: إذا اردت الوصول الى $A[i]$ فالعنوان يكون $base+4*i$

تمرين 2:

اكتب التعليمة اللازمة لحفظ محتوى $\$s0$ الجديد في الذاكرة عند العنوان $0x1001000c$ باعتبار العنوان القاعدي $\$t0$

الحل:

```
sw $s0,12($t0)    #M[0x1001000C] ← $s0
sw $s0,0xC($t0)    #M[0x1001000C] ← $s0
```

إذا هنا نحتاج لتعليمة حفظ أي sw وبما ان العنوان هو $0x1001000C$ والعنوان القاعدي $\$t0$ من الطلب السابق هو $0x10010000$ فنلاحظ ان القيمة التي جمعت له هي C اذا القيمة العددية الـ offset هي C وهي 12 في النظام العشري.

تمرين 3:

ماذا ينتج عند تنفيذ التعليمة التالية: $lw \$t1, 10(\$t0)$

سيينتج خطأ محاذاة (Alignment Error) بسبب طلب تحميل كلمة من عنوان ليس من مضاعفات العدد 4

تمرين 4:

المطلوب كتابة برنامج بلغة التجميع MIPS لجمع مصفوفة مؤلفة من 3 عناصر (3 كلمات) عنوان بداية المصفوفة في $\$t0$ مع حفظ الناتج النهائي في $\$t1$ (بدون استخدام الحلقات).

الحل:

```
lw $t2, 0($t0)
lw $t3, 4($t0)
lw $t4, 8($t0)
add $t1, $t2, $t3
add $t1, $t1, $t4
```

بداية قمنا بتخزين قيم العنوان في السجل القاعدي مع الازاحة باستخدام التعليمة lw ومن ثم قمنا بجمع قيم السجل $\$t2$ والسجل $\$t3$ ووضع القيمة في السجل $\$t1$ ومن ثم جمعنا قيمة السجل $\$t1$ الجديدة مع السجل $\$t4$ ووضعها في $\$t1$

تمرين 5:

بفرض ان محتوى $\$t0$ هو $0x1001000C$ حدد العنوان الذي تم التحميل منه الى $\$t4$

$0x10010014$

الحل:

إذاً لتحديد العنوان الذي تم التحميل منه نجمع السجل القاعدي مع مقدار الإزاحة و من الطلب الأول مقدار الإزاحة هو 8 (نحوه طبعاً للنظام الستة عشري وهنا هو نفسه ثمانية) نجمعه مع محتوى \$t0 فينتج المطلوب.
تذكرة الجمع في النظام الستة عشري +8 C الـ C وهي الرقم 12 نجمعها مع الثمانية فينتج 20 نطرح من العشرين 16 فيبقى أربعة ونرفع الرقم الآخر للأس واحد.

تمرين 6:

اكتب التعليمة اللازمة لتخزين \$t1 في عنوان الذاكرة 0x1001001C (باستخدام العنوان القاعدي \$t0 مع انزياح ممثل بالعشري/بالبست عشري)

الحل:

```
sw $t1, 16($t0)
sw $t1, 0x10($t0)
```

إذاً نعلم أن تعليمة التخزين هي sw العنوان القاعدي من الطلبات السابقة 0x1001000C لاحظ أن القيمة التي نخزنها كاملة أي العنوان هو 0x1001001C أي جمع للسجل القاعدي العدد 10 بالبست عشري وهو 16 بالعشري اذا نكتب التعليمات كما موضح اعلاه.

التعليمات الزائفة:

هي تعليمات يتم إتاحتها للمبرمج للتسهيل عليه و هي ليست موجودة في الـ hardware, إذاً فهي تعليمات ليست موجودة في المعالج فعلياً ولا يتم تنفيذها عتادياً وإنما يتولى المجمع assembler تحويلها الى تعليمات حقيقية مكافئة بهدف التسهيل على المبرمج, ومثال عنها التعليمات la - li - bgt - blt - bge - ble - mul - move وغيرها

- ملاحظة: السجل \$1 أو \$at (Assembler Temporary) محجوز للمجمع Assembler للتحويل من تعليمات زائفة الى تعليمات الآلة.

التعليمة الزائفة move :

```
move $t2, $t1
```

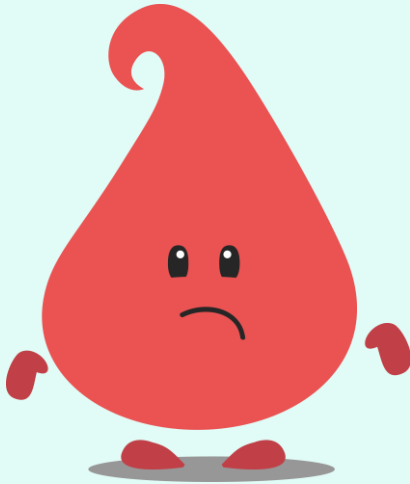
وهي تقوم بنسخ محتوى السجل \$t1 الى السجل \$t2
أي بفرض ان قيم السجلات الاولى هي: \$t2:20 , \$t1:10
بعد تنفيذ التعليمة تصبح القيم: \$t2:10 , \$t1:10

تمرين: اكتب التعليمة الحقيقية Real المكافئة للتعليمة الزائفة Pseudo التالية:

```
add $10,$0,$9      ↔      move $t2,$t1
```

- ملاحظة: يمكن مكافئة هذه التعليمة الزائفة بتعليمات اخرى مثل or او غيرها. وفي برنامج المحاكاة MARS نجد أن التعليمة الحقيقية المكافئة لتعليمة move السابقة هي:
addu \$10,\$0,\$9

وهي تعليمة تمنع ال overflow الفيض الذي من الممكن ان يحدث من عملية الجمع اذا بفرض انه يتم التخزين على 8 bit وفي جمع العددين:



```

8bit      1  0  0  0  0  0  0  0
+
8bit      1  0  0  0  0  0  0  1
-----
9bit     1  0  0  0  0  0  0  1
  
```

- اذا كنا نستخدم عملية الجمع العادية فسيعطي البرنامج رسالة بوجود overflow ويتوقف, لكن اذا استخدمنا addu فانها ستقوم بتجاهل الخانات الزائدة وتخزن الناتج.
- (أي انها ستخزن العدد 0000 0001 وتتجاهل اخر خانة وهي ال 1)
- تعليمة addu قد تؤدي الى وجود نتائج خاطئة في البرنامج, ولكنها تمنع توقفه بسبب ال overflow.

التعليمة الزائفة (li(load immediate) :

وهي تقوم بشحن سجل بقيمة عددية معينة مثل:

```
li $s1, 0x5FA2
```

اذا كانت القيمة العددية مؤلفة من 16bit او اقل li تكافئ تعليمة حقيقية واحدة وهي:

```
ori $s1, $0, 0x5FA2
```

- ملاحظة: التعليمة ori من النمط I-type لكن التعليمة li ليس لها نمط لانها تعليمة زائفة

كما يمكن استخدام التعليمة li لشحن بقيمة عددية ممثلة على 32 بت وهنا هذه التعليمة الزائفة ستكافئ تعليمتين حقيقيتين وهما:

السجل المحجوز لل Assembler

القسم العلوي من العدد

```
lui $1, upper(imm)
```

```
ori $s1,$1,lower(imm)
```

القسم السفلي من العدد

السجل المراد شحنه بالقيمة العددية

حيث تقوم التعليمة lui بتخزين القسم العلوي من العدد بالقسم العلوي من احد السجلات (وهنا استخدمنا السجل \$1 لاننا نمثل عمل ال assembler والذي يستخدم السجل \$1 لعملياته فقط) وهي من نمط I-type

■ ملاحظة: القسم السفلي للعدد هو اول 4 خانات، والقسم العلوي هو الخانات الباقية، مثال:



مثال: ما هي التعليمة الزائفة والتعليمتين الحقيقيتين المكافئتين لها لشحن السجل \$t0 بالعدد 0x14FF99؟

الحل:

```
li $t0, 0x14FF99
lui $at, 0x14          #$at=00140000
ori $t0, $at, 0xFF99   #0x00140000 or 0000FF99=0x0014FF99
```

■ ملاحظة: يمكن استخدام التعليمة الزائفة لـ شحن السجل القاعدي بقيمة العنوان المؤلف من 32 bit

تمارين

ضع القيمة 0xABCE000 في موقع الذاكرة ذي العنوان 0x10010004 باستخدام تعليمات حقيقية فقط.

الحل:

```
lui $t0, 0x1001
lui $t1, 0xABCD
ori $t1, $t1, 0xE000
sw $t1, 0x0004($t0)
```

إذا هدفنا هو وضع عنوان كامل مكون من 32 بت في تعليمة واحدة وهذا غير ممكن إلا باستخدام التعليمتين الحقيقيتين lui و ori إذا أولاً جهزنا العنوان القاعدي ف استخدمنا التعليمة الاولى

```
lui $t0, 0x1001
```

هذه التعليمة قامت بتحميل القيمة 0x1001 في النصف العلوي (اول 16 بت) من السجل \$t0 وستملأ النصف السفلي اصفاراً

لماذا لم نضع العنوان كامل؟ أي ان هناك القسم السفلي به العدد اربعة لما لم نضع التعليمة الاخرى وذلك لاننا اعتمدنا على الازاحة (offset) في تعليمة التخزين للوصول للعنوان النهائي 0x10010004 ثم قمنا ببناء القيمة المراد تخزينها وبما انها قيمة كبيرة 0xABCE000 (اكثر من 16 بت) لا يمكن تحميلها بتعليمة واحدة فاستخدمنا تعليمتين:

```
lui $t1, 0xABCD
ori $t1, 0xE000
```

إذا أصبح السجل \$t1 يحتوي على القيمة الكاملة
ثم عملية التخزين فاستخدمنا التعليمة

```
sw $t1, 0x0004($t0)
```

فجمعت قيمة السجل القاعدي مع قيمة الإزاحة المذكورة قبل السجل القاعدي لتحديد العنوان النهائي. وبالتالي تم تنفيذ المطلوب.

تمرين: بفرض لدينا محتوى الذاكرة الموضح بالشكل، اكتب التعليمات اللازمة لحفظ العدد 500 في موقع الذاكرة ذي العنوان 0x10010014 باعتبار العنوان القاعدي t0=0x10010000

الحل:

```
li $t0, 0x10010000 #lui, ori
li $t1, 500 #ori
sw $t1, 0x14($t0) #sw $t1, 20($t0)
```

- 1- اكتب التعليمات اللازمة لحفظ بقية الاعداد في مواقع الذاكرة المحددة بالشكل (بحيث يكون مقدار الإزاحة OFFSET ممثلاً بالست عشري في تعليمات التخزين SW ثم قم بتحميل اخر عنصرين من المصفوفة واجمعهما، بحيث تستخدم مقدار الإزاحة بالتمثيل العشري في تعليمات التحميل.
- 2- ما هو عنوان ذاكرة الخلية؟ قم بتخزين العدد 50 في بطريقتين: مرة بانزياح صفر، و مرة اعتماداً على العنوان القاعدي في \$t0.

0x1001 0000	20
0x1001 0004	40
0x1001 0008	80
0x1001 000C	200
0x1001 0010	300
0x1001 0014	500
0x1001 0018	600
0x1001 001C	10
0x1001 0020	5

```
li $t0, 0x10010000 # lui , ori
li $t1, 500 # ori
sw $t1, 0x14($t0) # sw $t1, 20($t0)
```

عنوان ذاكرة الخلية المخزن فيها العدد 500 هو ناتج جمع: 0x10010000 بالعمود أيسر الخلية مع مقدار الانزياح +14 أعلى الخلية بـ hex، أي: 0x10010014

ذاكرة المعطيات في برنامج المحاكاة MARS

Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)
0x10010000	20	40	80	200	300	500	600	10
0x10010020	5	0	0	0	0	0	0	0
0x10010040	0	0	0	0	0	0	0	0
0x10010060	0	0	0	0	0	0	0	0
0x10010080	0	0	0	0	0	0	?	0
0x100100a0	0	0	0	0	0	0	0	0

وظيفة: ١- اكتب التعليمات اللازمة لحفظ بقية الأعداد في مواقع الذاكرة المحددة بالشكل (بحيث يكون مقدار الإزاحة OFFSET ممثلاً بالست عشري في تعليمات التخزين SW، ثم قم بتحميل آخر عنصرين من المصفوفة واجمعهما، بحيث تستخدم مقدار الإزاحة بالتمثيل العشري في تعليمات التحميل.

٢- ما هو عنوان ذاكرة الخلية ؟ ، قم بتخزين العدد 50 في بطريقتين: مرة بانزياح صفر، ومرة اعتماداً على العنوان القاعدي في \$t0.

الحل:

بداية سنحمل العنوان القاعدي في \$t0 وهو معطى بالقيمة 0x10010000

```
li $t0, 0x10010000
```

سنخزن العدد 20 (مقدار الإزاحة 0x0)

```
li $t1, 20
sw $t1, 0x0($t0)
```

تخزين العدد 40 مقدار الإزاحة 0x4

```
li $t1, 40
sw $t1, 0x4($t0)
```

تخزين العدد 80 (مقدار الازاحة 0x8)

```
li $t1, 80
sw $t1, 0x8($t0)
```

تخزين العدد 200 (مقدار الازاحة 0xC)

```
li $t1, 200
sw $t1, 0xC($t0)
```

تخزين العدد 300 (مقدار الازاحة 0x10)

```
li $t1, 300
sw $t1, 0x10($t0)
```

تخزين العدد 500 (مقدار الازاحة 0x14)

```
li $t1, 500
sw $t1, 0x14($t0)
```

تخزين العدد 600 (مقدار الازاحة 0x18)

```
li $t1, 600
sw $t1, 0x18($t0)
```

تخزين العدد 10 (مقدار الازاحة 0x1C)

```
li $t1, 10
sw $t1, 0x1C($t0)
```

تخزين العدد 5 (مقدار الازاحة 0x20)

```
li $t1, 5
sw $t1, 0x20($t0)
```

المطلوب هو تحميل اخر عددين وهما 5 و 10 وجمعهما باستخدام مقدار الازاحة بالنظام العشري
لدينا العنصر قبل الاخير (10) عنوانه 0x1001001C الازاحة 1C بالسبت عشري وهو 28 بالعشري
العنصر الاخير (5) عنوانه 0x10010020 الازاحة 20 وهي 32 بالعشري

```
lw $s0, 28($t0)
lw $s1, 32($t0)
add $s2, $s0, $s1
```

الطلب الثاني: عند تحديد موقع اشارة الاستفهام نجد انها تقع عند الصف الذي عنوانه 0x10010080
العمود الذي يمثل الازاحة (+18) value

■ ملاحظة الازاحة في MARS تكتب بالسبت عشري اذا الازاحة هي 0x18

ومنه فان العنوان هو $0x10010080 + 0x18 = 0x10010098$

ومن اجل تخزين العدد 50 في هذا العنوان بطريقتين

الطريقة الاولى: بانزياح (ازاحة) صفر:

اذا هنا نضع العنوان كاملا في سجل ثم نستخدم ازاحة 0

```
li $t1, 50
li $t2, 0x10010098
sw $t1, 0($t2)
```

الطريقة الثانية: اعتماداً على العنوان القاعدي في \$t0 نحن نعلم ان \$t0 يحتوي علي 0x10010000 لذا نستخدم الازاحة للوصول للخلية المطلوبة (انزياح السطر (0x80) + انزياح العمود (0x18)) = 0x98

```
li $t1, 50
sw $t1, 0x98($t0)
```

بفرض ان المحتوى الابتدائي لكل سجل هو رقم السجل x 2 (مثلاً 6 = \$3)

والمحتوى الابتدائي لكل موقع في الذاكرة هو قيمة العنوان x 100 (مثلاً Mem[25]=2500) حدد عمل كل تعليمة من التعليمات التالية:

1. lw \$13, 32(\$10)
2. sw \$7, 48(\$2)
3. lw \$15, 16(\$16)
4. sw \$18, 0x14(\$8)

الحل:

1. العملية التي لدينا هي تحميل كلمة من الذاكرة و وضعها في السجل \$13
لحساب العنوان: نرى انه العنوان القاعدي الموجود في السجل \$10

اذا قيمة السجل $10 \times 2 = 20$

مقدار الازاحة هو 32

عنوان الذاكرة المستهدف $52 = 32 + 20$

ومنه فان القيمة المحملة حسب الفرضية محتوى الذاكرة في العنوان 52 هو:

$Mem[52] = 52 \times 100 = 5200$

النتيجة النهائية: سينم وضع القيمة 5200 داخل السجل \$13

2. العملية هنا هي تخزين كلمة من السجل \$7 الى الذاكرة

القيمة المراد تخزينها هي محتوى السجل \$7 قيمته $7 \times 2 = 14$

حساب العنوان القاعدي موجود في السجل \$2

قيمة السجل $2 \times 2 = 4$

مقدار الازاحة هو 48

عنوان الذاكرة المستهدف $52 = 48 + 4$

النتيجة النهائية: سيتم تحديث محتوى موقع الذاكرة 52 ليصبح 14 (أي ستصبح $Mem[52] = 14$ بدلا من قيمتها السابقة)

3. العملية هنا هي تحميل كلمة من الذاكرة و وضعها في السجل \$15

لحساب العنوان: نرى انه العنوان القاعدي الموجود في السجل هو \$16

اذا قيمة السجل $16 \times 2 = 32$

مقدار الازاحة هو 16

عنوان الذاكرة المستهدفة $48 = 32 + 16$

القيمة المحملة: حسب الفرض محتوى الذاكرة في العنوان 48 هو: $Mem[48] = 48 \times 100 = 4800$



النتيجة النهائية: سيتم وضع القيمة 4800 داخل السجل \$15

4. العملية: تخزين كلمة من السجل \$18 الى الذاكرة

القيمة المراد تخزينها: هي محتوى السجل \$18 قيمته $18 \times 2 = 32$

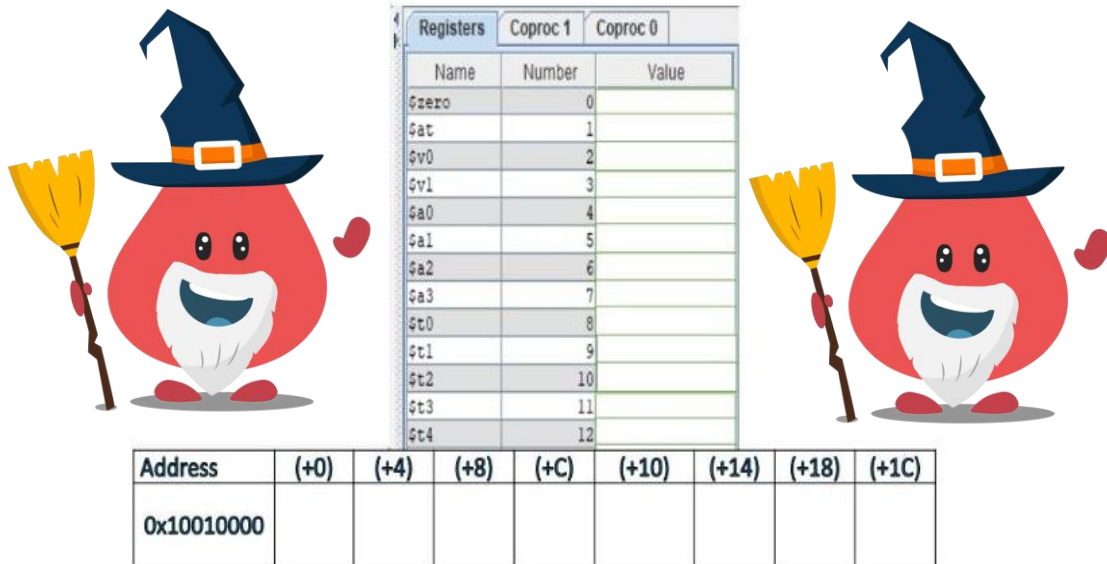
حساب العنوان: العنوان القاعدي موجود في السجل \$8

مقدار الازاحة هنا مكتوب بالست عشري 0x14 ولحسابه نحوله للعشري: $4 + (16 \times 1) = 20$

عنوان الذاكرة المستهدف = $36 = 20 + 16$

النتيجة النهائية: سيتم تحديث محتوى موقع الذاكرة 36 ليصبح 36

اضف التعليمات المناسبة لتوضيح عمل التعليمات ثم استنتج محتوى الذاكرة والسجلات عند انتهاء تنفيذ البرنامج، وتأكد من صحة الحل بالاستعانة ببرنامج المحاكاة MARS .



Registers	Coproc 1	Coproc 0
Name	Number	Value
\$zero	0	
\$at	1	
\$v0	2	
\$v1	3	
\$a0	4	
\$a1	5	
\$a2	6	
\$a3	7	
\$t0	8	
\$t1	9	
\$t2	10	
\$t3	11	
\$t4	12	

Address	(+0)	(+4)	(+8)	(+C)	(+10)	(+14)	(+18)	(+1C)
0x10010000								

```
li $t0, 0x10010000 #
addi $t1, $zero, 6#
addi $t2, $zero, 9#
sw $t1, 8($t0) #
sw $t2, 12($t0) #
sub $t2, $t2, $t1 #
sw $t2, 4($t0) #
lw $t3, 12($t0) #
and $t2, $t3, $t2 #
sw $t2, 16($t0) #
```

Registers	Coproc 1	Coproc 0
Name	Number	Value
\$zero	0	0x00000000
\$at	1	0x10010000
\$v0	2	0x00000000
\$v1	3	0x00000000
\$a0	4	0x00000000
\$a1	5	0x00000000
\$a2	6	0x00000000
\$a3	7	0x00000000
\$t0	8	0x10010000
\$t1	9	0x00000006
\$t2	10	0x00000001
\$t3	11	0x00000009
\$t4	12	0x00000000

الحل:

Address	Value (+0)	Value (+4)	Value (+8)	Value (+C)	Value (+10)	Value (+14)
0x10010000	0x00000000	0x00000003	0x00000006	0x00000009	0x00000001	0x00000000
0x10010020	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010040	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010060	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000

تمرين للتدريب والتطبيق على برنامج المحاكاة

اكتب برنامجا بلغة التجميع MIPS يقوم بها يلي (باستخدام تعليمات حقيقية حصرا):

شحن السجلين \$t0 \$t1 بعددين ممثلين بالست عشري احدهما من 8 بت والاخر مؤلف من 20 بت واكتب في التعليقات المكافئ الزائف لهذه التعليمات pseudo instructions

خزن محتوى السجلين السابقين في الذاكرة في موقعين متتاليين بدءا من العنوان القاعدي 0x1001004C احسب ضعفي القيمتين السابقتين، ثم احفظ الناتجين عند عنواني الذاكرة التاليين: 0x10010068، 0x1001008C قم بتحميل القيم الاربعة السابقة من الذاكرة الى السجلات \$t4,\$t5,\$t6,\$t7 واجمعهم بحيث يكون ناتج جمع هذه السجلات في \$t4 ثم احفظ هذا الناتج في الذاكرة عند الموقع ذي الانزياح offset=16 من العنوان 0x100100a4 واكتب في التعليق التعليمة وفيها الانزياح offset ممثلا بالنظام الست عشري اشر الى قيم السجلات النهائية و مواقع الذاكرة التي تم تعديلها (مع التعليق)

الحل:

```
ori $t0,$0,0xAC #li $t0,0xAC
lui $t1,0x000A
ori $t1,$t1,0x1BF4
#li $t1,0x1BF4
lui $1,0x1001
ori $s0,$1,0x004C
sw $t0,0x0($s0)
sw $t1,0x4($s0)
add $t2,$t0,$t0
add $t3,$1,$t1
lui $s1,0x1001
```

```
add $t4,$t4,$t7
lui $s3,0x1001
ori $s3,$s3,0x00A4
sw $t4,0x10($s3)
```

```
ori $s1,$s1,0x0068
sw $t2,0($s1)
lui $s2,0x1001
ori $s2,$s2,0x008C
sw $t3,0($s2)
lw $t4,0($s0)
lw $t5,0x4($s0)
lw $t6,0x0($s1)
lw $t7,0x0($s2)
add $t4,$t4,$t5
add $t4,$t4,$t6
```



Execute

Text Segment

Bkpt	Address	Code	Basic	Source
	0x00400000	0x340800ac	ori \$8,\$0,0x000000ac	1: ori \$t0,\$0,0xac \$t0,0xac
	0x00400004	0x3c09000a	lui \$9,0x0000000a	2: lui \$t1,0x000a
	0x00400008	0x35291bf4	ori \$9,\$9,0x00001bf4	3: ori \$t1,\$t1,0x1bf4 \$t1,0x1bf4
	0x0040000c	0x3c011001	lui \$1,0x00001001	4: lui \$1,0x1001
	0x00400010	0x3430004c	ori \$16,\$1,0x0000004c	5: ori \$s0,\$1,0x004c
	0x00400014	0xae080000	sw \$8,0x00000000(\$16)	6: sw \$t0,0x0(\$s0)
	0x00400018	0xae090004	sw \$9,0x00000004(\$16)	7: sw \$t1,0x4(\$s0)
	0x0040001c	0x01085020	add \$10,\$8,\$9	8: add \$t2,\$t0,\$t0
	0x00400020	0x00295820	add \$11,\$1,\$9	9: add \$t3,\$1,\$t1
	0x00400024	0x3c111001	lui \$17,0x00001001	10: lui \$s1,0x1001
	0x00400028	0x36310068	ori \$17,\$17,0x00000068	11: ori \$s1,\$s1,0x0068
	0x0040002c	0xae2a0000	sw \$10,0x00000000(\$...	12: sw \$t2,0(\$s1)

Data Segment

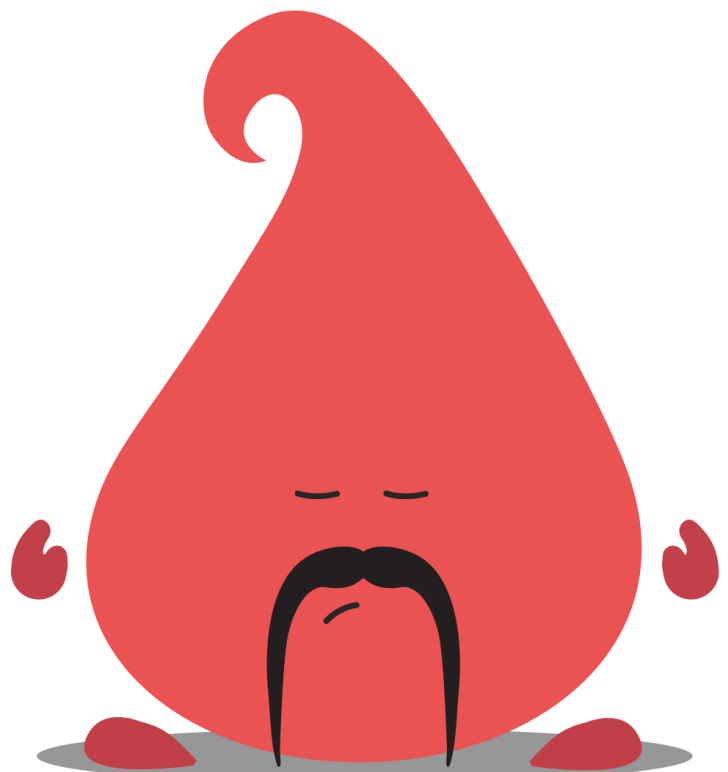
Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)
0x10010000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010020	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010040	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010060	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010080	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100a0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100c0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100e0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010100	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010120	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000

Registers Coproc 1 Coproc 0

Name	Number	Value
\$zero	0	0x00000000
\$at	1	0x00000000
\$0	2	0x00000000
\$v1	3	0x00000000
\$a0	4	0x00000000
\$a1	5	0x00000000
\$a2	6	0x00000000
\$a3	7	0x00000000
\$t0	8	0x00000000
\$t1	9	0x00000000
\$t2	10	0x00000000
\$t3	11	0x00000000
\$t4	12	0x00000000
\$t5	13	0x00000000
\$t6	14	0x00000000
\$t7	15	0x00000000
\$s0	16	0x00000000
\$s1	17	0x00000000
\$s2	18	0x00000000
\$s3	19	0x00000000
\$s4	20	0x00000000
\$s5	21	0x00000000
\$s6	22	0x00000000
\$s7	23	0x00000000
\$s8	24	0x00000000
\$s9	25	0x00000000
\$k0	26	0x00000000
\$k1	27	0x00000000
\$gp	28	0x10008000
\$sp	29	0x7fffffc0
\$fp	30	0x00000000
\$ra	31	0x00000000
\$pc		0x00400000
\$hi		0x00000000
\$lo		0x00000000

Mars Messages Run I/O

Discipline is choosing
between what you
want now and what
you want **most** (:



أسئلة للمحاضرة الثانية عملي :

1-الكلمة الواحدة تتكون من:

- a - 4 بايتات
- b - 8 بايتات
- c - 16 بايت
- d - 4 بتات

الإجابة : a

2- في العنوان بالبايتات, فإنه لا يمكن الوصول للمحارف داخل الكلمة (صح أم خطأ)

الإجابة : خطأ

3-تبعد كل تعليمة عن التي تسبقها بـ:

- a - 4 بايتات
- b - 8 بايتات
- c - 16 بايت
- d - 4 بتات

الإجابة : a

4-إذا افترضنا أن هناك تعليمة في العنوان 100, فإن التعليمة التي تليها (في حال كانت العنوان بالكلمة) يكون عنوانها:

- a - 101
- b - 102
- c - 104
- d - 108

الإجابة : a

5-يميل مبرمجو لغة Assembly إلى استخدام الذاكرة للعمليات الحسابية والمنطقية بدلاً من السجلات (صح أم خطأ)

الإجابة : خطأ

6-إذا كان عدد خطوط العنوان 32 خطأً, فكم عدد العناوين التي يمكن تخزينها في الذاكرة

- a - 2^{32}
- b - 2×32
- c - $(2^{32}) - 1$
- d - لا شيء مما سبق

الإجابة : a

7- لنفترض أن هناك مصفوفة A تبدأ من العنوان $x100100000$ والتي يتم حفظها في المسجل $t0$ ، ما هي التعليمات الصحيحة لقراءة العنصر الخامس في المصفوفة وحفظه في المسجل $s0$ ؟

a- $sw \$s0, 20(\$t0)$

b- $sw \$s0, 16(\$t1)$

c- $lw \$s0, 16(\$t1)$

d- $lw \$s0, 0x10(\$t0)$

الإجابة : d

تذكير: "للوصول إلى عنوان العنصر (i) في مصفوفة $A[i]$ ، نستخدم الصيغة التالية:

الأساس $+ 4 * i$ "

8- اكتب التعليمات التي تخزن قيمة السجل $s0$ في العنوان $0x1001000C$ ، مع الأخذ في الاعتبار أن العنوان الأساسي ($x100100000$) موجود في $t0$.

$sw \$s0, 0xC(\$t0)$

أو

الإجابة : $sw \$s0, 12(\$t0)$

9- ما هي التعليمات الصحيحة لتحميل قيمة من العنوان $0x1001001C$ وتخزينها في السجل $v0$ والعنوان الأساسي في $s0$ ؟

a- $lw \$v0, 12(\$s0)$ حيث $\$s0$ على $\$s0$

b- $lw \$v0, 0x8(\$s0)$ حيث $\$s0$ على $\$s0$

c- $sw \$v0, 28(\$s0)$ حيث $\$s0$ على $\$s0$

d- $lw \$v0, 0x18(\$s0)$ حيث $\$s0$ على $\$s0$

الإجابة : d

10- ماهي التعليمة/ات الحقيقية المطابقة للتعليمة الزائفة $li \$s1, 0x00005FA2$:

a- $ori \$s1, \$s0, 0x5FA2$

b- $andi \$s1, \$0, 0x5FA2$

c- $lui \$at, 0x0000$

$ori \$s1, \$at, 0x5FA2$

d- $ori \$s1, \$zero, 24482$

الإجابة : c

11- لماذا يضطر المبرمج لاستخدام الذاكرة بدلاً من الاعتماد الكلي على السجلات (Registers) في معالج MIPS ؟

a- لأن السجلات بطيئة جداً في تنفيذ العمليات.

b- لأن حجم السجلات محدود (32 سجلاً فقط) وقد لا تتسع لكمية البيانات الكبيرة.

c- لأن الذاكرة تعتبر أقل تكلفة من حيث زمن الوصول (Access time).

d- لأن معالج MIPS لا يدعم العمليات الحسابية داخل السجلات.

الإجابة : b

12- في معالج MIPS ماهو نوع العنوان المستخدم ، وكيف يتم حساب المسافة بين كل تعليمة والتي تليها ؟

- a- العنوان بالكلمة، وتبعد كل تعليمة عن الأخرى بمقدار 1 بايت.
- b- العنوان بالبايت، وتبعد كل تعليمة عن الأخرى بمقدار 1 بايت.
- c- العنوان بالبايت، وتبعد كل تعليمة عن الأخرى بمقدار 4 بايت.
- d- العنوان بالكلمة، وتبعد كل تعليمة عن الأخرى بمقدار 4 بايت

الإجابة : c

13- ما هي وظيفة عداد البرنامج (PC - Program Counter) وكيف يتغير عند الانتقال من تعليمة إلى أخرى؟

- a- يحمل قيمة البيانات التي يتم معالجتها، ويزداد بمقدار 1.
- b- يُوْشِر على التعليمة الحالية (عنوانها) ويزداد بمقدار 4 للانتقال للتعليمة التالية.
- c- يخزن عدد التعليمات التي تم تنفيذها منذ بداية التشغيل.
- d- يحمل عنوان السجل القاعدي (Base register) ويزداد بشكل عشوائي.

الإجابة : b

14- عند تنفيذ تعليمة التحميل lw \$t1, 10(\$t0) ، ما هي النتيجة المتوقعة؟

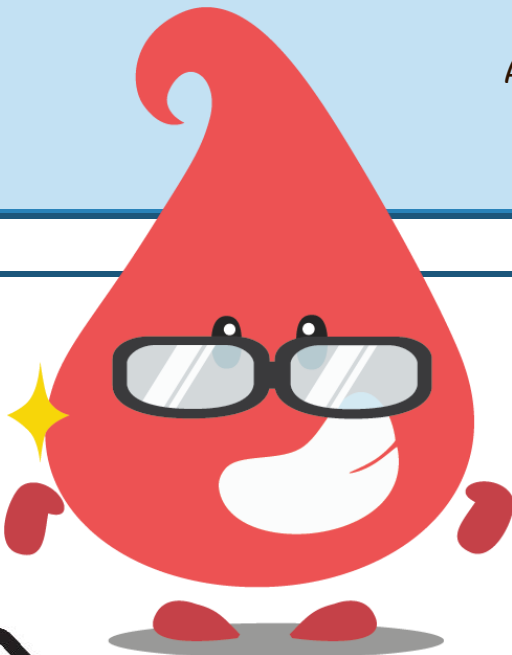
- a- سيتم تحميل البيانات من العنوان الناتج عن جمع محتوى \$t0 مع الرقم 10 بنجاح .
- b- سينتج "خطأ محاذاة (Alignment Error)" لأن عنوان تحميل الكلمة ليس من مضاعفات العدد 4
- c- سيتم تحميل 10 بايتات من الذاكرة إلى السجل \$t1
- d- سيتم تخزين محتوى السجل \$t1 في العنوان 10.

الإجابة : b

15- ما هو السجل الذي يحجزه المجمع (Assembler) بشكل خاص ليستخدمه في عملية تحويل التعليمات الزائفة إلى تعليمات حقيقية؟

- a- السجل \$zero (رقم 0)
- b- السجل \$at (رقم 1) المسمى Assembler Temporary
- c- السجل \$ra (رقم 31)
- d- السجل \$gp (رقم 28)

الإجابة : b



16-التعليمات الزائفة (Pseudo-instructions) هي تعليمات:

- a- تم تنفيذها عتادياً (Hardware) في المعالج بشكل مباشر
- b- توفر لتسهيل البرمجة، ويقوم المجمع (Assembler) بتحويلها لتعليمات حقيقية مكافئة
- c- مخصصة فقط للتعامل مع العمليات الحسابية المعقدة جداً
- d- لا يمكن استخدامها في برنامج المحاكاة MARS

الإجابة : b

17-لشحن سجل بقيمة عددية كبيرة (32 بت) باستخدام التعليمات الحقيقية، نحتاج لاستخدام:

- a-تعليمية la واحدة فقط
- b-تعليمية lw مع إزاحة كبيرة
- c-تعليمتين حقيقيتين هما lui و ori
- d-تعليمية add مع السجل \$zero.

الإجابة: c

18-ما الفرق الجوهرى بين تعليمية add وتعليمية addu في لغة تجميع MIPS ؟

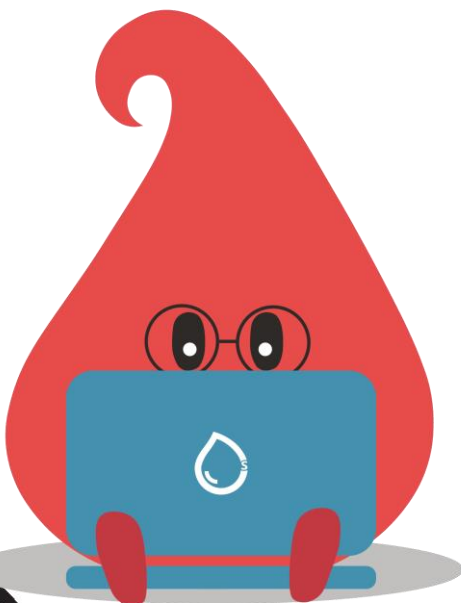
- a - addu تستخدم فقط مع الأعداد السالبة
- b - addu (تمنع توقف البرنامج عند حدوث "فيض (Overflow) " عن طريق تجاهل الخانات الزائدة)
- c - add أسرع بكثير من addu في التنفيذ
- d - لا يوجد فرق بينهما وكلاهما يعطي نفس النتيجة دائماً.

الإجابة : b

19- ماهو نوع التعليمية la ؟

- a - R-type
- b - I-type
- c - J-type
- d - ليس لها نوع

الإجابة : d



To be so lucky that God makes you responsible for changing the destiny of an entire family :)